

# Computation Offloading and Resource Allocation in Vehicular MEC: A Parameterized Deep Reinforcement Learning Approach

Ruofei Ma<sup>1</sup>, Member, IEEE, Jingyang Zhou<sup>2</sup>, Ruisong Wang<sup>3</sup>, Hsiao-Hwa Chen<sup>4</sup>, Life Fellow, IEEE, and Gongliang Liu<sup>5</sup>, Member, IEEE

**Abstract**—Rapid advancement of the Internet of vehicles (IoV) technology has created a lot of intelligent vehicle applications, such as real-time navigation, collision avoidance, and autonomous driving, etc., which in turn generates a significant amount of computation-intensive tasks. Due to limited computation capabilities of in-vehicle devices, mobile edge and cloud computing are viewed as promising solutions to offload latency-sensitive tasks to mobile edge computing servers to meet the requirements for eURLLC in 5G/6G applications. Vehicular edge computing systems require effective offloading decision-making and resource management to maximize resource utilization. However, existing scheduling schemes suffer a high complexity to deal with multi-dimensional resource optimization problems, thereby introducing a significant latency and compromising resource utilization efficiency. This paper aims to address the issues on latency, energy consumption, and cost jointly within a vehicular edge computing system. In particular, a multi-objective optimization problem is formulated for its joint optimization on computation offloading and resource allocation with the help of a deep reinforcement learning (DRL) approach. To this end, a parameterized deep Q-networks (P-DQN) algorithm is introduced for solving a binary offloading problem. Extensive numerical simulations validate the superiority of the proposed scheme compared to well-known algorithms in the literature.

**Index Terms**—Internet of Vehicles (IoV), mobile edge computing, task offloading, deep reinforcement learning, parameterized deep Q-network.

## I. INTRODUCTION

THE Internet of vehicles (IoV), as a form of the Internet of things (IoT) tailored for 5G and beyond systems, emerges

Received 14 August 2024; revised 22 January 2025 and 30 March 2025; accepted 20 May 2025. Date of publication 29 May 2025; date of current version 20 November 2025. This work was supported in part by Shandong Provincial Natural Science Foundation under Grant ZR2023MF001 and Grant ZR2024QF140, in part by the National Natural Science Foundation of China under Grant 61801144 and Grant 61971156, and in part by the Taiwan Ministry of Science and Technology under Grant 113-2221-E-006-143 and Grant 113-2221-E-006-144. The review of this article was coordinated by Dr. Ebrahim Bedeer. (Corresponding author: Hsiao-Hwa Chen.)

Ruofei Ma, Jingyang Zhou, Ruisong Wang, and Gongliang Liu are with the Department of Communication Engineering, Harbin Institute of Technology, Weihai 264209, China (e-mail: maruofei@hit.edu.cn; zjyfs@163.com; mathwrs@163.com; liugl@hit.edu.cn).

Hsiao-Hwa Chen is with the Department of Engineering Science, National Cheng Kung University, Tainan City 70101, Taiwan (e-mail: hshwchen@ieee.org).

Digital Object Identifier 10.1109/TVT.2025.3574783

as a critical component in future intelligent transportation systems (ITS) [1], [2], [3]. Via interconnecting numerous network entities within transportation systems, such as vehicles, roadside units (RSUs), and pedestrians, IoV is able to offer driving assistance/safety assurance and deliver other life-critical information to drivers and passengers. The advancement of smart vehicles has created numerous smart vehicle applications, including augmented reality, real-time navigation, voice/image/video recognition, and autonomous driving, focusing primarily on safety issues [4], [5], [6], [7]. These applications will generate computation-intensive tasks requiring high reliability and low latency. However, the limited computation capacity of in-vehicle devices can hardly meet the intensive computation demands of these applications [8], [9], [10].

Mobile edge computing (MEC) technology, which integrates computing resources and intelligence at the network edges, offers a promising solution to address these challenges by providing fast and efficient computation services to vehicles [11], [12], [13]. Offloading computing tasks to MEC servers close to the terminals can effectively deal with the problem of insufficient in-vehicle computing resources. Despite its numerous advantages, edge computing technology faces various application challenges [14]. For instance, network operators are required to pre-deploy a large number of RSUs and edge servers. In addition, designing efficient computation offloading and resource allocation algorithms is essential to enhance resource utilization and computation efficiency of edge computing systems. The key to ensure vehicular MEC system's quality of service (QoS) and overall performance is to jointly optimize vehicles to edge servers task offloading policy, energy consumption, and multi-dimensional resource allocation [15], [16], [17].

To tackle the challenges associated with MEC, many works have been done in recent years. A substantial amount of literatures focused on devising optimal policies for task offloading and resource management under various requirements, aiming to optimize latency, energy consumption, or/and QoS. Fan et al. [18] focused on joint task offloading and resource allocation issues for vehicular edge computing using both V2I and V2V communications. Aiming to minimize task processing delays, they formulated the joint scheduling issue as an optimization problem, in which task diversity, vehicle classification, and processing flexibility were considered. They used generalized

benders decomposition (GBD) and reformulation linearization (RL) to derive the optimal solution and designed a heuristic scheme to obtain sub-optimal solutions with a lower complexity. Liu et al. [19] converted an offloading decision-making problem into a joint delay and energy consumption optimization problem and introduced a joint computing offloading and energy optimization scheme. This scheme merged a game-theory-based offloading strategy with a convex optimization-based resource allocation algorithm to balance latency and energy consumption in vehicular edge computing systems. Luo et al. [20] depicted a computation offloading problem as a distributed offloading decision game to achieve a Nash equilibrium via a distributed self-learning algorithm. In [21], the authors proposed binary and partial reverse offloading strategies for diverse tasks, aiming at alleviating the burden on edge servers and reducing system latency through optimal utilization of constrained vehicular computing resources. In [22], a heuristic scheme was suggested to address joint latency and energy consumption optimization challenge in vehicular fog computing. Wang et al. [23] proposed a two-stage optimization scheme, combining a Lyapunov-based approach for computation offloading and resource allocation with a BAS-Elman neural network for lightweight target tracking. This method effectively balances UAV energy consumption, tracking success rate, and data processing accuracy under stochastic conditions. Du et al. [24] proposed an SDN-based hierarchical framework for hybrid edge and cloud computing systems, introducing evolutionary and Stackelberg differential games to model the interactions between users, edge computing providers (ECPs), and cloud computing providers (CCPs). This framework enables dynamic resource pricing and allocation strategies by considering users' service preferences and the supply-demand relationship between ECPs and CCPs. Simulations demonstrated the proposed mechanism's ability to improve resource utilization and service quality while achieving faster convergence compared to static approaches. Nevertheless, the majority of the existing schemes relied on convex optimization and game theory. The algorithms based on convex optimization typically suffer high computational complexities and the game-theoretic schemes necessitate multiple iterations to achieve the Nash equilibrium [25], [26], [27], complicating their practical applications in dynamic vehicular networks. On the other hand, heuristic algorithms are usually less complex, but they likely tend to converge on local (rather than global) optimal solutions [28].

The advancement of artificial intelligence (AI) has introduced new solutions for resource management in wireless communications [29], [30], [31]. In particular, deep reinforcement learning (DRL), which integrates deep learning (DL) and reinforcement learning (RL), is not contingent upon the environment model, rendering it in particular suitable for the applications in dynamic environments. Inspired by the use of DRL in dynamic control scenarios, several works concentrated on developing DRL-based computation offloading strategies [32], [33], [34]. The Deep Q-Network (DQN) represents an amalgamation of Q-learning and DL, employing deep neural networks to approximate the Q-function and thus broadening Q-learning's applicability in continuous state spaces. Liu et al. [35] devised a vehicular

edge computing network architecture, where idle vehicles served as edge servers and the optimization problem of maximizing vehicular edge computing network's long-term utility was formulated as a semi-Markov decision process. The authors utilized Q-learning and DQN to propose two computation offloading and resource allocation strategies. Wang et al. [36] addressed the challenges of joint scheduling on task offloading, resource allocation, and service caching in a multi-server three-tier MEC system. A Double DQN (DDQN) based algorithm called SCRACO was proposed to dynamically and efficiently manage offloading decisions and allocate communication and computing resources. Another DRL scheme, namely deep deterministic policy gradient (DDPG) algorithm, works based on actor-critic approach with DQN. Peng et al. [37] employed DDPG and a hierarchical RL framework to tackle multi-dimensional resource optimization challenge in multi-access edge computing. They investigated the issues on allocations of spectrum, computation, and storage resources jointly, aiming to maximize QoS in vehicular applications. Yang et al. [38] considered an edge-terminal machine learning task paradigm in vehicular networks to evaluate latency, processing accuracy, and energy consumption. A two-stage DRL-based computation offloading and resource allocation strategy was proposed, aiming at identifying optimal solutions for vehicle offloading decision and transmit power allocation. Xue et al. [39] explored joint management of offloading, computation, and storage resources in vehicular edge computing to minimize system's task processing latency, based on which a centralized DRL algorithm was proposed. Ma et al. [40] developed a vehicle trajectory prediction model based on deep neural networks and integrated Lyapunov optimization with DRL techniques to maximize task offloading throughput with a long-term stability of task queue. Geng et al. [41] concentrated on optimizing latency and energy consumption by partitioning computing tasks into sub-tasks and modeling them as directed acyclic graphs (DAGs). They proposed a distributed computing offloading method (DCOM) to leverage multi-agent DRL. Chen et al. [42] proposed a multi-task-driven framework for user association and resource allocation in heterogeneous in-vehicle networks (HetIVNs). They developed a Proximal Policy Optimization (PPO) based algorithm to optimize task success rates and minimize WiFi access point energy consumption, achieving a superior efficiency compared to baseline methods. In addition, the introduction of multi-modal learning and aggregation techniques showed a significant potential to enhance MEC systems. Du et al. [43] explored distributed training of foundation models (FMs) in 6G systems, leveraging federated learning (FL) and over-the-air computation (AirComp) to address challenges such as data heterogeneity and communication instability. These techniques enable efficient aggregation of real-time data from vehicular networks, improving resource allocation and task offloading strategies. For instance, the use of pipeline parallelism and adaptive communication resource allocation supports scalable and low-latency model update, making these methods well-suited for high-dynamic environments of vehicular MEC.

In this paper, we consider a binary offloading problem in vehicular edge computing networks, which is typically formulated

as a mixed integer nonlinear programming (MINLP) problem. As a non-convex problem, it is highly complex in nature. When DRL is applied to it, it necessitates the management of a hybrid action space, which integrates both discrete and continuous actions. The existing studies normally **addressed the hybrid action space challenge in joint optimization problems by either discretizing continuous actions or relaxing discrete action spaces**. However, such approaches exacerbated the complexity of the problem and sacrifice the accuracy of the solutions. In response to the aforementioned challenges, this work **utilizes parameterized DRL to tackle a real-time offloading decision-making problem**. The objective herein is to minimize system's long-term latency, energy consumption, and cost. Specifically, the main contributions of this work can be summarized as follows.

- We consider a cloud-assisted vehicular edge computing scenario by constructing a three-layer task offloading architecture, which encompasses local computation, edge server, and cloud layers. This architecture helps to address the challenges associated with resource constraints at edge servers and the imbalance of computing resources in vehicular edge computing environments.
- An optimization problem is formulated to jointly consider latency, energy consumption, and cost under a Markov decision process (MDP) framework. **We design state space and reward function and introduce a parameterized action space to effectively manage the mixed action space**. Furthermore, an algorithm based on the parameterized deep Q-network (P-DQN) is proposed to address the optimization problem.
- The efficacy of the proposed scheme is validated through extensive numerical simulations, which indicate that the proposed scheme secures a higher reward in comparison to the baseline algorithms.

The rest of this paper is outlined as follows. Section II introduces a system model for a vehicular MEC network. In Section III the optimization problem for joint computation offloading and resource allocation is formulated. Section IV depicts the proposed RL based approach to address the optimization problem. Simulations and performance evaluation are provided in Section V, followed by the conclusions in Section VI.

## II. SYSTEM MODEL

In this section, a system model of vehicular MEC is introduced. A network model for vehicular MEC is provided first, followed by computation task and communication models. Specifically, computation task processing models at local vehicle, MEC server, and cloud server are detailed. For analytical clarity, the symbols used in subsequent discussions are summarized in Table I.

### A. Network Model

As illustrated in Fig. 1, a vehicular MEC computing network comprises  $N$  vehicles,  $M$  uniformly distributed RSUs, and a cloud server. The sets of vehicles and RSUs are denoted as  $\mathcal{N} = \{1, 2, \dots, N\}$  and  $\mathcal{M} = \{1, 2, \dots, M\}$ , respectively. Each RSU is equipped with an MEC server having a computing

TABLE I  
MAJOR SYMBOLS AND THEIR DEFINITIONS

| Notation           | Description                                                                               |
|--------------------|-------------------------------------------------------------------------------------------|
| $\mathcal{N}$      | A set of vehicles                                                                         |
| $\mathcal{M}$      | A set of RSUs with MEC servers                                                            |
| $N$                | The number of vehicles                                                                    |
| $M$                | The number of RSUs/MEC servers                                                            |
| $i$                | Index of vehicles                                                                         |
| $m$                | Index of RSUs/MEC servers                                                                 |
| $t$                | Index of time slots                                                                       |
| $\bar{\mu}$        | Average speed of each vehicle                                                             |
| $\bar{\sigma}$     | Standard deviation of vehicles' speed                                                     |
| $x_i$              | Position of vehicle $i$                                                                   |
| $R$                | Coverage range of each RSU                                                                |
| $r_{R2R}$          | Transmission rate between RSUs                                                            |
| $F$                | Computation capacity of each MEC server                                                   |
| $F^c$              | Computation capacity of cloud server                                                      |
| $f_i^l$            | Computation capacity of vehicle $i$                                                       |
| $\mathcal{T}_i(t)$ | Task generated by vehicle $i$ at time slot $t$                                            |
| $d_i(t)$           | Size of input data of $\mathcal{T}_i(t)$                                                  |
| $c_i(t)$           | Computation intensity of $\mathcal{T}_i(t)$                                               |
| $t_i^{finish}(t)$  | Maximum tolerated delay of $\mathcal{T}_i(t)$                                             |
| $s_i^d(t)$         | Offloading indicator for vehicle $i$ at time slot $t$                                     |
| $a_i(t)$           | Offloading decision for $\mathcal{T}_i(t)$                                                |
| $f_{i,m}^E(t)$     | Computation resources that MEC server $m$ allocates to vehicle $i$ for $\mathcal{T}_i(t)$ |
| $f_i^c(t)$         | Computation resources that cloud server allocated to vehicle $i$ for $\mathcal{T}_i(t)$   |
| $h_{i,m}$          | Channel gain between vehicle $i$ and RSU $m$                                              |
| $p_i(t)$           | Transmit power of vehicle $i$ at time slot $t$                                            |
| $\rho^E, \rho^c$   | Unit price of MEC/cloud server's computation resources                                    |
| $t^c$              | Upload time for input data from RSU to cloud server                                       |
| $t^{inter}$        | Transmission delay between adjacent RSUs                                                  |
| $t_i^{link}(t)$    | Remaining time that vehicle $i$ stays on road                                             |
| $T_i^{max}(t)$     | Actual tolerance delay for $\mathcal{T}_i(t)$                                             |
| $w_1, w_2, w_3$    | Weights for delay, energy consumption, and cost                                           |

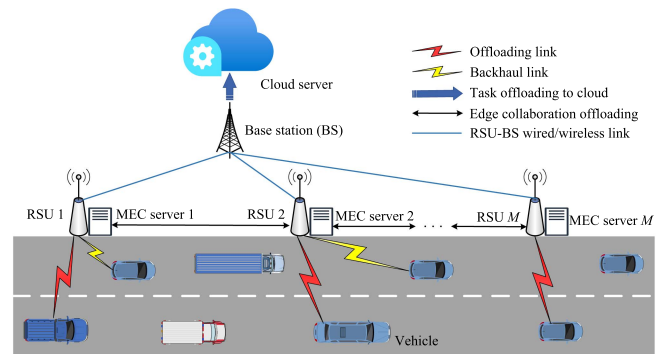


Fig. 1. A network model for vehicular mobile edge computing.

capacity of  $F$  and has an identical coverage range  $R$ . These RSUs are connected to a remote cloud server via wired connections and core network. The road is divided into  $M$  segments with vehicles randomly distributed over them, i.e., each RSU covers a particular road segment. The length of road is denoted as  $L$ .

RSUs equipped with MEC servers are interconnected via wired connections and can offer reliable and real-time task offloading services for vehicles within their coverage areas. Each vehicle connects to only an RSU serving the road segment. **The computation tasks generated by a vehicle can be processed locally on in-vehicle device or via offloading them to corresponding RSU MEC or cloud servers.** If task offloading is performed, a vehicle's task can be offloaded to its directly connected RSU, adjacent RSUs, or a cloud server. Note that offloading task to adjacent RSUs or cloud servers requires relay assistance from the RSU directly connected to the vehicle, and such situations may happen when a vehicle's directly connected RSU is overloaded and unable to process computing tasks, which is helpful to mitigate imbalance issue in using edge computing resources. The cloud server with abundant computing resources is capable of handling high-complexity tasks or serving as a supplement when network computing resources are insufficient. Additionally, time is divided into equal-length slots denoted as  $t = \{1, 2, \dots, T\}$ , assuming each vehicle generates at most one task per slot, with each slot having a uniform duration  $\Delta t$ .

In this model, vehicle speeds are assumed to follow a truncated Gaussian distribution, reflecting speed randomness with non-negative values. The probability density function (PDF) of the truncated Gaussian distribution is given by

$$\hat{f}(v) = \begin{cases} \frac{2f(v)}{\text{erf}\left(\frac{v_{\max}-\bar{\mu}}{\sqrt{2}\bar{\sigma}}\right) - \text{erf}\left(\frac{v_{\min}-\bar{\mu}}{\sqrt{2}\bar{\sigma}}\right)}, & V_{\min} \leq v \leq V_{\max}, \\ 0, & \text{otherwise,} \end{cases} \quad (1)$$

where  $f(v) = \frac{1}{\sqrt{2\pi}\bar{\sigma}} \exp(-\frac{(v-\bar{\mu})^2}{2\bar{\sigma}^2})$  represents PDF of Gaussian distribution, with  $\bar{\mu}$  and  $\bar{\sigma}$  denoting average and standard deviation of vehicle speed, respectively.  $V_{\max} = \bar{\mu} + 3\bar{\sigma}$  and  $V_{\min} = \bar{\mu} - 3\bar{\sigma}$  denote maximal and minimal vehicular speeds, respectively, and  $\text{erf}(\bar{x}) = \frac{2}{\sqrt{\pi}} \int_0^{\bar{x}} e^{-z^2} dz$  is an error function.

### B. Task Model

To characterize the computation tasks of vehicles, the following task model is presented. For vehicle  $i$ , the computation task generated during time slot  $t$  is typically represented as a triplet, i.e.,

$$\mathcal{T}_i(t) = \{d_i(t), c_i(t), t_i^{\text{finish}}(t)\}, \quad (2)$$

where  $d_i(t)$  denotes input data size for computation task  $\mathcal{T}_i(t)$ , and  $c_i(t) = \lambda_i(t) \cdot d_i(t)$  represents computation intensity of  $\mathcal{T}_i(t)$ , i.e., the number of CPU cycles needed to complete the task. Here,  $\lambda_i(t)$  is a coefficient describing the relationship between  $d_i(t)$  and  $c_i(t)$ , termed as service coefficient and determined by task's computation complexity.  $t_i^{\text{finish}}(t)$  denotes maximum tolerable delay of task  $\mathcal{T}_i(t)$ .

Based on the framework as described above, let us consider a three-tier task computation model encompassing local processing, edge offloading, and cloud offloading. Vehicles can choose to either process computation tasks locally or offload them to MEC servers or cloud servers. For each computation task, a vehicle may offload it to at most one server. While vehicles can only communicate directly with an RSU in their proximity area, they

can offload tasks through this RSU to MEC servers via neighboring RSUs. Thus, for vehicle  $i$ , the set of possible offloading decisions for task  $\mathcal{T}_i(t)$  during time slot  $t$  includes  $M + 2$  options, denoted by  $a_i(t) = \{0, 1, 2, \dots, M + 1\}$ , where  $a_i(t) = 0$  represents local execution, and  $a_i(t) = M + 1$  denotes cloud offloading with  $a_i(t) = m$  ( $1 \leq m \leq M$ ) signifying offloading to MEC server  $i$ . Furthermore, for vehicle  $i$ , offloading data are defined as  $\mathcal{S} = \{s_i^d(t) \mid i \in \mathcal{N}, d \in \{l, E, c\}, s_i^d(t) \in \{0, 1\}\}$ .  $\mathcal{D} = \{l, E, c\}$  represents a set of offloading policies. Specifically,  $s_i^l(t) = 1$  when  $a_i(t) = 0$ , indicating local processing, and  $s_i^l(t) = 0$  otherwise;  $s_i^E(t) = 1$  for  $a_i(t) = m$  ( $1 \leq m \leq M$ ), indicating offloading to MEC server  $i$ , and  $s_i^E(t) = 0$  otherwise;  $s_i^c(t) = 1$  when  $a_i(t) = M + 1$ , indicating offloading to cloud, and  $s_i^c(t) = 0$  otherwise.

### C. Communication Model

When a vehicle decides to offload a task, it must upload task's input data to the corresponding RSU and receive the output data from the RSU upon task completion. Next, we introduce a vehicle-RSU communication model based on vehicle-to-infrastructure (V2I) communications. Assume that uplink channel from vehicle  $i$  to RSU  $m$  is a Rayleigh fading channel without co-channel interference. The channel gain is expressed as

$$h_{i,m} = K_0 g_{i,m} \bar{g}_{i,m} L_{i,m}^{-\eta}, \quad (3)$$

where  $K_0$  is a constant corresponding to the path loss model,  $L_{i,m}$  is the distance between vehicle  $i$  and RSU  $m$ , and  $\eta$  is the path loss exponent.  $g_{i,m}$  denotes a small-scale fast fading gain following an exponential distribution with unit mean.  $\bar{g}_{i,m}$  denotes a slow fading gain following a log-normal distribution with its standard deviation of 8 dB.

Assume that total uplink bandwidth available for vehicles to each RSU is  $B$ , and at a given time slot  $t$ ,  $\bar{N}(t)$  vehicles need to offload their tasks to RSU  $m$ . Under these condition, uplink data rate from vehicle  $i$  to RSU  $m$  can be calculated as

$$r_{i,m}(t) = \frac{B}{\bar{N}(t)} \log_2 \left( 1 + \frac{p_i(t) h_{i,m}(t)}{\sigma^2} \right), \quad (4)$$

where  $p_i(t)$  is the transmit power of vehicle  $i$  at time slot  $t$ ,  $h_{i,m}(t)$  represents channel gain between vehicle  $i$  and RSU  $m$  at time slot  $t$ , and  $\sigma^2$  denotes background noise.

### D. Task Processing Model

Based on the previous network and task models, this subsection introduces a task processing model for various task execution strategies (i.e., local processing, MEC processing, and cloud processing) chosen by vehicles.

1) *Local Processing Model*: In the local computing approach, vehicles use onboard computing resources to process computation tasks. Assume vehicle  $i$ 's computing capacity is denoted as  $f_i^l$ . The time to complete task  $\mathcal{T}_i(t)$  locally is calculated as

$$T_i^l(t) = \frac{c_i(t)}{f_i^l}. \quad (5)$$

The energy consumption for the local processing strategy is given by

$$E_i^l(t) = \kappa \cdot c_i(t)(f_i^l)^2, \quad (6)$$

where  $\kappa$  is an energy consumption parameter dependent on chip architecture used [44].

Moreover, this paper evaluates the cost associated with offloading tasks in vehicular edge computing environments. If tasks are processed locally, vehicles incur no costs for computing resources from server operators. Thus, we can denote the cost for vehicle  $i$  as

$$\epsilon_i^l(t) = 0. \quad (7)$$

**2) MEC Processing Model:** If a computation task is offloaded to an MEC server, its input data must be uploaded first via vehicle's uplink to a directly connected RSU. Then, if the target MEC is available on an RSU directly connected to the source vehicle, the task will be executed directly. As each RSU is equipped with an MEC server, transmission time between a RSU and its equipped MEC server can be omitted. But if the target MEC is not on a vehicle's directly connected RSU, this RSU will relay those input data to the target RUS/MEC (directly or with other RSUs' assistance) for final task execution. Thus, if vehicle  $i$  is within the coverage area of RSU  $m'$ , the time required for vehicle  $i$  to upload its offloading task's input data to its target MEC server  $m$  can be written as

$$T_{i,m}^{up}(t) = \frac{d_i(t)}{r_{i,m'}(t)} + t^b, \quad (8)$$

where  $t^b = t^{que} + \frac{d_i(t)}{r_{R2R}}$  denotes total delay for data delivery between RSUs when  $m' \neq m$ . Here,  $t^{que}$  is queuing delay at the RSU for waiting for other tasks' input data to be sent, and  $\frac{d_i(t)}{r_{R2R}}$  denotes transmission delay when sending data from RSU  $m'$  to RSU  $m$ .  $r_{R2R}$  represents data rate between RSUs. When  $m' = m$ , we have  $t^b = 0$ . Assume that computation tasks are executed in parallel on an MEC server. The computing resources requested by vehicle  $i$  from MEC server  $m$  are denoted as  $f_{i,m}^E(t)$ . Consequently, execution time and cost for offloading task  $\mathcal{T}_i(t)$  to MEC server  $m$  are calculated respectively as

$$T_{i,m}^{exe}(t) = \frac{c_i(t)}{f_{i,m}^E(t)}, \quad (9)$$

$$\epsilon_{i,m}^E(t) = \rho^E f_{i,m}^E(t), \quad (10)$$

where  $\rho^E$  represents the unit price for computation resources of a MEC server.

After completing task execution on MEC server  $m$ , computation results are sent back to vehicle via RSU's downlink. As a task's result has normally a relatively small data size, delay and energy consumption for directly transmitting the result back can be neglected. However, considering dynamic nature of high-speed vehicles, if vehicle  $i$  exits the coverage area of the corresponding RSU, computation result must be transferred to the RSU serving the road segment that vehicle  $i$  is currently running on via inter-RSU communication before downloading

the results. Therefore, the total transmission time for sending the computation result back to the vehicle can be calculated as

$$T_{i,m}^{down}(t) = \Delta x \cdot t^{inter}, \quad (11)$$

where  $\Delta x$  represents the number of road segments between vehicle  $i$  and RSU  $m$ , and  $t^{inter}$  denotes the transmission delay for sending computation result data between two adjacent RSUs, decided primarily by propagation delay, which is a small value. Let  $x_i$  denote the position of vehicle  $i$  on road and  $k$  indicate that the vehicle is within the coverage area of the  $k$ th RSU. Then,  $k = \lfloor \frac{x_i}{R} \rfloor$  and  $\Delta x = |k - m|$ , with  $\lfloor \cdot \rfloor$  being the floor function. Finally, energy consumption and total delay for MEC server processing model are calculated as

$$E_{i,m}^E(t) = p_i(t)T_{i,m}^{up}(t), \quad (12)$$

$$T_{i,m}^E(t) = T_{i,m}^{up}(t) + T_{i,m}^{exe}(t) + T_{i,m}^{down}(t). \quad (13)$$

**3) Cloud Processing Model:** If a computation task is offloaded to a cloud server, task's input data is uploaded first from the vehicle to the RSU it is connected, and then from the RSU to a cloud server. However, as data transmission between the cloud server and RSU involves long-distance communications, it is necessary to account for the upload time from the RSU to the cloud server and the time for data transmission back from the cloud server to the RSU, regardless of the data volume of computation results. Consequently, total delay for processing decisions by the cloud server is

$$T_{i,m}^c(t) = T_{i,m}^{up}(t) + \frac{c_i(t)}{f_i^c(t)} + 2t^c, \quad (14)$$

where  $f_i^c(t)$  represents the computation resources allocated by the cloud server to vehicle  $i$ , and  $t^c$  denotes a constant data transmission time between the RSU and the cloud server. Assume that  $\rho^c$  denotes the unit price for cloud server's computation resources. Energy consumption and cost for the cloud server processing model are expressed as

$$E_{i,m}^c(t) = p_i(t)T_{i,m}^{up}(t), \quad (15)$$

$$\epsilon_{i,m}^c(t) = \rho^c f_i^c(t). \quad (16)$$

Based on the aforementioned models, we can go ahead to formulate an optimization problem in the next section.

### III. PROBLEM FORMULATION

In this section, we formulate a task offloading and resource allocation optimization problem for the proposed vehicular MEC network, aiming to minimize system's long-term latency, energy consumption, and cost. Based on the analysis given in Section II, we can define execution delay, energy consumption, and cost for task  $\mathcal{T}_i(t)$  as

$$T_i(t) = s_i^l(t)T_i^l(t) + s_i^E(t)T_{i,m}^E(t) + s_i^c(t)T_{i,m}^c(t), \quad (17)$$

$$E_i(t) = s_i^l(t)E_i^l(t) + s_i^E(t)E_{i,m}^E(t) + s_i^c(t)E_{i,m}^c(t), \quad (18)$$

$$\epsilon_i(t) = s_i^l(t)\epsilon_i^l(t) + s_i^E(t)\epsilon_{i,m}^E(t) + s_i^c(t)\epsilon_{i,m}^c(t). \quad (19)$$

Our objective is to design a joint computation offloading and resource allocation strategy to minimize the delay, energy consumption, and cost for vehicular computation tasks. Consequently, a utility function for vehicle  $i$  can be defined as

$$U_i(t) = w_1 \frac{T_{\max} - T_i(t)}{T_{\max}} + w_2 \frac{E_{\max} - E_i(t)}{E_{\max}} - w_3 \frac{\epsilon_i(t)}{\epsilon_{\max}}, \quad (20)$$

where  $w_1$ ,  $w_2$ , and  $w_3$  are the weights for delay, energy consumption, and cost, respectively, with the constraint  $0 \leq w_1, w_2, w_3 \leq 1$  and  $w_1 + w_2 + w_3 = 1$ . In this paper, we limit the maximum computation resource allocated to vehicles by the cloud server to be  $F^c$ . Therefore,  $T_{\max} = \max t_i^{\text{finish}}(t)$ ,  $E_{\max} = \max\{\max \kappa \cdot c_i(t)(f_i^l)^2, T_{\max} \cdot P_{\max}\}$ , and  $\epsilon_{\max} = \max\{\rho^E \cdot F, \rho^c \cdot F^c\}$  represent the worst-case delay, energy consumption, and cost, respectively.

Although maximum tolerable delay  $\mathcal{T}_i(t)$  is  $t_i^{\text{finish}}(t)$ , if a vehicle exits the entire area, i.e., leaves the coverage areas of all RSUs, offloaded task's results may not be transmitted back via interconnected RSUs. Let  $t_i^{\text{link}}$  denote the remaining time over which vehicle  $i$  stays on the road; thus actual delay tolerance for task  $\mathcal{T}_i(t)$  is  $T_i^{\max}(t) = \min\{t_i^{\text{finish}}(t), t_i^{\text{link}}(t)\}$ . Based on the system model, offloading decisions for all vehicles are defined as  $\mathcal{A} = \{a_i(t) | i \in \mathcal{N}, t \in \{1, 2, \dots, T\}\}$ . Additionally, services are defined as the computation resources allocated to vehicles and the corresponding transmit powers, which are denoted as  $\mathcal{F} = \{f_{i,m}^E(t), f_i^c(t) | i \in \mathcal{N}, m \in \mathcal{M}, t \in \{1, 2, \dots, T\}\}$  and  $\mathcal{P} = \{p_i(t) | i \in \mathcal{N}, t \in \{1, 2, \dots, T\}\}$ , respectively. To maximize system's long-term utility, the optimization problem can be formulated as

$$J = \max_{\mathcal{A}, \mathcal{F}, \mathcal{P}} \frac{1}{T} \sum_{t=1}^T \sum_{i=1}^N U_i(t), \quad (21)$$

$$\text{s.t. } s_i^l(t) + s_i^E(t) + s_i^c(t) = 1, \forall i, t, \quad (21a)$$

$$s_i^d(t) = \{0, 1\}, d \in \{l, E, c\}, \forall i, t, \quad (21b)$$

$$a_i(t) \in \{0, 1, \dots, M+1\}, \forall i, t, \quad (21c)$$

$$T_i(t) \leq T_i^{\max}(t), \forall i, t, \quad (21d)$$

$$0 \leq f_{i,m}^E(t) \leq F, 0 \leq \sum_{i=1}^N f_{i,m}^E(t) \leq F, \forall t, m, \quad (21e)$$

$$0 \leq f_i^c(t) \leq F^c, \forall i, t, \quad (21f)$$

$$P_{\min} \leq p_i(t) \leq P_{\max}, \forall i, t. \quad (21g)$$

The above problem aims to minimize system's long-term delay, energy consumption, and cost via making optimal offloading decisions  $\mathcal{A}$  and resource allocations  $\mathcal{F}$  and  $\mathcal{P}$ . Constraints (21a) and (21b) ensure that each vehicle selects only one offloading option, i.e., local, MEC server, or cloud server. (21c) defines decision variables, indicating the tasks to be offloaded to a server or executed locally. (21d) details the time constraints for task execution. (21e) indicates that resources allocated to vehicles by an MEC server must not exceed its total computation capacity. (21f) sets a upper limit for resources that a cloud server can allocate to each vehicle without exceeding cloud server's total capacity. (21g) ensures transmit power of each vehicle not exceeding the maximum allowed.

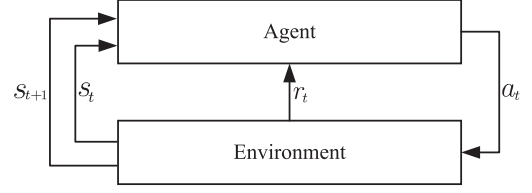


Fig. 2. The framework of the DRL algorithm.

Analyzing problem (21), we can identify that it is a mixed-integer nonlinear programming (MINLP) problem, which is difficult to solve with most conventional methods. Convex optimization solutions suffer a high computation complexity and cannot adapt to vehicular networks' rapidly changing environment, hindering adaptive task offloading. To tackle this challenge, this paper proposes to use a DRL-based joint scheduling task offloading and resource allocation algorithm. In particular, to overcome the limitations in the previous works, which normally required relaxation or discretization of variables, we introduce a parameterized action space for handling mixed discrete-continuous variables.

#### IV. REINFORCEMENT LEARNING BASED APPROACH

In this section, we will convert the formulated optimization problem to an MDP and propose a new DRL based solution. An MDP is characterized by tuple  $(\mathcal{S}, \mathcal{A}, P, r)$ , where  $\mathcal{S}$  denotes a set of system states or a state space,  $\mathcal{A}$  represents a set of actions, and  $r(s | a)$  is the reward function dependent on both state and action, which can be simplified to  $r(s)$  if only the state influences the reward.  $P(s' | s, a)$  is a state transition function, indicating the probability of transferring to state  $s'$  after action  $a$  is executed in state  $s$ . However, the state transition function is often unknown due to the lack of prior environmental knowledge, complicating the process to find an optimal solution via MDP dynamic programming. Therefore, DRL is applied to address the formulated optimization problem. Fig. 2 illustrates the fundamental structure of the DRL approach, where the agent aims to maximize long-term cumulative reward. Initially, the agent observes state  $s_t$  from the environment and selects action  $a_t$  based on the current policy. Following the action, the system transitions to state  $s_{t+1}$ , returning reward  $r_t$  to the agent, which then updates its policy based on this feedback, ultimately converging to the optimal policy.

##### A. State, Action and Reward of DRL Algorithm

To better illustrate the proposed approach, in this subsection, we will define the state space, action space, and reward for the DRL algorithm based on the proposed system model.

1) *State Space*: The state space should be constructed first. For task offloading decision and resource allocation, it is crucial to know servers' computation resource distribution, input data sizes of all computation tasks, and changing location information of vehicles. Therefore, at time slot  $t$ , system state  $s_t \in \mathcal{S}$  consists of five components as

$$s_t = \{\mathbf{F}_m(t), \mathbf{F}_i^l(t), \mathbf{X}(t), \mathbf{V}(t), \mathbf{D}(t)\}. \quad (22)$$

Detail explanations for the state vector  $s_t$  are given as follows.

- $\mathbf{F}_m(t) = [f_1(t), f_2(t), \dots, f_M(t)]$  signifies the current distribution of computing resources across different edge nodes.
- $\mathbf{F}_i^l(t) = [f_1^l(t), f_2^l(t), \dots, f_N^l(t)]$  represents available computational resources for each vehicle at the current time.
- $\mathbf{X}(t)$  and  $\mathbf{V}(t)$  depict the mobility of all vehicles, with  $\mathbf{X}(t)$  and  $\mathbf{V}(t)$  denoting vehicles' positions and velocities at time slot  $t$ , respectively. This information indicates the urgency of computation tasks and the selection of edge nodes for offloading. Specifically, the state vectors detailing vehicles' location information are defined as

$$\mathbf{X}(t) = [x_1(t), x_2(t), \dots, x_i(t) \cdots, x_N(t)], \quad (23)$$

$$\mathbf{V}(t) = [v_1(t), v_2(t), \dots, v_i(t) \cdots, v_N(t)]. \quad (24)$$

- $\mathbf{D}_i(t) = [d_1(t), d_2(t), \dots, d_i(t) \cdots, d_N(t)]$  specifies input data sizes for all tasks awaiting for execution. The input data sizes of tasks directly impact decision-making on where and how to offload the tasks, affecting both transmission times and computation resource requirements.

Note that vehicle  $i$  cannot observe the entire state space. The observation space of vehicle  $i$  is defined as  $o_i(t) = \{\mathbf{F}_m(t), f_i^l(t), x_i(t), v_i(t), \mathbf{D}(t)\}$ . Vehicle  $i$  can obtain information about input data sizes of other vehicles from interconnected RSUs. The observation space of a vehicle corresponds to the input state of the corresponding agent.

2) *Action Space*: Upon observing state  $o_i(t)$ , vehicle  $i$  needs to make an offloading decision. The dimension of action vector varies according to offloading decision, which can be expressed as

$$\mathbf{a}_i(t) = \begin{cases} [a_i(t)], & a_i(t) = 0, \\ [a_i(t), f_{i,m}^E(t), p_i(t)], & a_i(t) \in \{1, 2, \dots, M\}, \\ [a_i(t), f_i^c(t), p_i(t)], & a_i(t) = M + 1, \end{cases} \quad (25)$$

where  $a_i(t)$  denotes offloading decision for vehicle  $i$ . As mentioned earlier,  $a_i(t) = 0$  implies local execution,  $a_i(t) > 0$  indicates offloading to a server, defined within the space  $\mathcal{A} = \{0, 1, \dots, M + 1\}$  and spanning over  $M + 2$  dimensions,  $f_{i,m}^E(t)$  and  $f_i^c(t)$  represent the resources that MEC server  $m$  and cloud server allocate to vehicle  $i$ , respectively, and  $p_i(t)$  denotes vehicle's transmit power.

To manage heterogeneous action vectors, a parameterized action space with a hierarchical structure is introduced [45]. Vehicles initially determine their discrete action  $a_i(t)$  and then select the associated continuous parameters  $y_{a_i}(t)$ , with  $y_{a_i}(t)$  being a vector of variable dimensions. Define  $\mathcal{Y}$  as a continuous set for all  $a_i(t) \in \mathcal{A}$ . Then, discrete-continuous hybrid action space  $\mathbb{A}$  can be expressed as

$$\mathbb{A} = \{(a_i(t), y_{a_i}(t)) \mid a_i(t) \in \mathcal{A}, y_{a_i}(t) \in \mathcal{Y}\}. \quad (26)$$

3) *Reward*: Agents adjust their policy based on the rewards they have received. In alignment with the objective function

of (21), the reward at time slot  $t$  is defined as

$$r_t = \begin{cases} U_i(t), & T_i(t) \leq T_i^{\max}(t), \\ R_p + w_2 E_i(t) - w_3 \epsilon_i(t), & \text{otherwise}, \end{cases} \quad (27)$$

where  $R_p$  denotes a penalty introduced to ensure that the agent's actions comply with constraint (21d). Furthermore, a long-term cumulative discounted reward can be expressed as

$$R_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots = \sum_{j=0}^{\infty} \gamma^j r_{t+j}, \quad (28)$$

where  $\gamma$  is a weight ranging between 0 and 1.

### B. Parameterized DRL Based Strategy

This subsection outlines a methodology to solve the MINLP problem in (21) utilizing DRL. Initially, we explored the most prevalent DQN and DDPG schemes in current literatures. Subsequently, a scheme based on parameterized deep Q-network (P-DQN) was proposed, where P-DQN merges the features of both DQN and DDPG to achieve more efficient management for hybrid action spaces. Note that the proposed schemes are implemented in each vehicle, and thus in the text followed, to simply the expressions of mathematical symbols, we will omit the subscript  $i$  (the index of vehicles) and put time slot number  $t$  at the subscript position in all relevant mathematical symbols, e.g.,  $a_i(t)$  and  $y_{a_i}(t)$  are simplified to  $a_t$  and  $y_{a_t}$ , respectively.

1) *DQN Based Solution*: DQN is a value-based reinforcement learning algorithm, which uses Q-function to describe a long-term cumulative discounted reward for an agent's action  $a_t$  in state  $s_t$ . The Q-function, first defined in Bellman equation, can be expressed as

$$\begin{aligned} Q(s, a) &= \mathbb{E}[R_t | s_t = s, a_t = a] \\ &= \mathbb{E} \left[ \sum_{j=0}^{\infty} \gamma^j r_{t+j} \mid s_t = s, a_t = a \right] \\ &= \mathbb{E} \left[ r_t + \gamma \max_{a_{t+1} \in \mathcal{A}} Q(s_{t+1}, a_{t+1}) \mid s_t = s, a_t = a \right]. \end{aligned} \quad (29)$$

DQN utilizes a neural network to approximate the Q-function instead of calculating the Q-value directly for each state-action pair. This approximation, i.e.,  $Q(s, a; \omega) \approx Q(s, a)$ , denotes  $\omega$  as network's weights. In addition, DQN calculates target temporal difference (TD) error using a network with weights  $\omega_{\text{tg}}$ , and it updates its parameters via a least square loss function as

$$L_t(\omega) = \left\{ Q(s_t, a_t; \omega) - [r_t + \gamma \max_{a_{t+1} \in \mathcal{A}} Q(s_{t+1}, a_{t+1}; \omega_{\text{tg}})] \right\}^2. \quad (30)$$

DQN synchronizes the target network parameters with the Q-network's parameters at predefined intervals, setting  $\omega_{\text{tg}} = \omega$ . Upon estimating  $Q^*$ , DQN adopts a greedy policy, always selecting an action with the maximum Q-value. However, in hybrid action spaces, DQN can approximate continuous parameter set

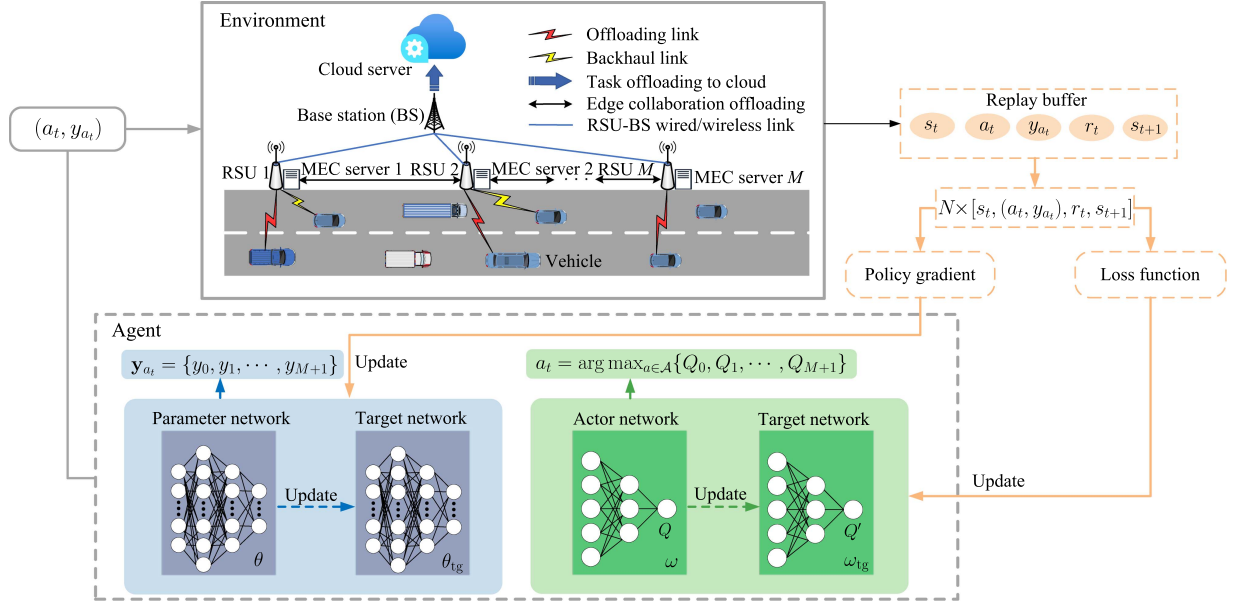


Fig. 3. The architecture of the P-DQN based solution for vehicular edge computing.

$\mathcal{Y}$  only with a discrete subset, which often needs numerous discrete actions to maintain the accuracy of solution.

2) *DDPG Based Solution:* Based on an actor-critic architecture, DDPG employs two neural networks to approximate deterministic policy function  $\mu(s; \theta)$  and value function  $Q(s, a; \omega)$ . To tackle the challenge inherent in deterministic policy algorithms, action noise is introduced during training. The actor network inputs state  $s_t$  and outputs action  $a_t = \mu(s_t; \theta)$ , corresponding to the current policy. Conversely, critic network evaluates action values. This requires to meet the following condition:

$$Q(s_t, \mu(s_t; \theta); \omega) \approx r(s_t, a_t) + \gamma Q(s_{t+1}, \mu(s_{t+1}; \theta_{tg}); \omega_{tg}), \quad (31)$$

where  $\omega_{tg}$  and  $\theta_{tg}$  represent the weights of target networks, which are utilized to calculate TD loss for updating critic network and policy gradient for updating actor network. The definition of the loss function is the same as (30) and the policy gradient can be computed as [46]

$$\nabla_{\theta} J = \mathbb{E} \left[ \nabla_{\theta} \mu_{\theta}(s) \nabla_a Q(s, a; \omega) |_{a=\mu_{\theta}(s)} \right], \quad (32)$$

whose explanation can be found in [46], and we will not repeat it here.

DDPG updates the weights of target networks using a soft updating algorithm as

$$\theta_{tg} = \tau_p \theta + (1 - \tau_p) \theta_{tg}, \quad (33)$$

$$\omega_{tg} = \tau_a \omega + (1 - \tau_a) \omega_{tg}, \quad (34)$$

where  $\tau_p$  and  $\tau_a$  are scale factors used to update  $\theta_{tg}$  and  $\omega_{tg}$ , respectively, with their values ranging in (0,1).

Since the deterministic policy function  $\mu(s; \theta)$  in DDPG produces continuous values, it cannot generate actions directly in

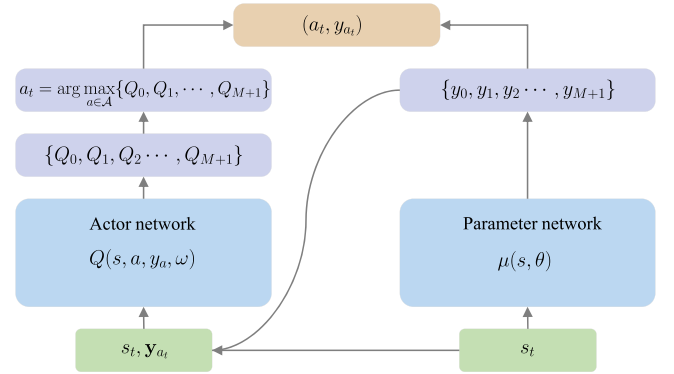


Fig. 4. The network structure of P-DQN.

a discrete-continuous hybrid action space. Therefore, the action space must be modeled as a continuous set. In [47], the authors defined an approximate action space. In the proposed scheme, DDPG uses an actor network to simultaneously output weight values for discrete actions and continuous actions. Subsequently, a  $\epsilon$ -greedy strategy selects discrete actions by choosing the highest-weight action with probability  $1 - \epsilon$ , where  $\epsilon$  is a small value to enable an exploration through occasional random action selection. After selecting a discrete action, the corresponding continuous action is chosen based on that discrete action's parameters. This approach complicates the structure of action space, hindering agent's convergence to the optimal policy.

3) *P-DQN Based Solution:* This paper presents a vehicular network task offloading and resource allocation algorithm utilizing P-DQN, and its architecture is depicted in Fig. 3. P-DQN is a DRL algorithm tailored for discrete-continuous hybrid action space, and is capable of efficiently dealing with the formulated optimization problem (21). Fig. 4 demonstrates a basic network structure of P-DQN, which combines the features

of both DQN and DDPG. Initially, deterministic policy network  $\mu(s, \theta)$  produces all continuous actions based on the current state  $s_t$ . Subsequently, state  $s_t$ , together with the values of continuous actions, is fed into the value network  $Q(s, a, y_a, \omega)$ . The algorithm subsequently selects a discrete action corresponding to the highest Q-value. Finally, the selected discrete action, in conjunction with its corresponding continuous parameters, constitutes an action vector.

In the P-DQN algorithm, Q-network is utilized to approximate action-value function based on Bellman equation, which is expressed in a hybrid action space as follows

$$\begin{aligned} Q(s_t, a_t, y_{a_t}, \omega) &\approx Q(s_t, a_t, y_{a_t}) = \\ \mathbb{E}_{r_t, s_{t+1}} \left[ r_t + \gamma \max_{a \in \mathcal{A}} \sup_{y_a \in \mathcal{Y}} Q(s_{t+1}, a, y_a) \mid s_t, (a_t, y_{a_t}) \right] \\ &= \mathbb{E}_{r_t, s_{t+1}} \left[ r_t + \gamma \max_{a \in \mathcal{A}} Q(s_{t+1}, a, y_a^Q(s_{t+1})) \mid s_t \right]. \end{aligned} \quad (35)$$

In (35),  $\arg \sup_{y_a \in \mathcal{Y}} Q(s, a, y_a)$  is represented by function  $y_a^Q : \mathcal{S} \rightarrow \mathcal{Y}$ . We approximate  $y_a^Q(s)$  using a deterministic policy network  $\mu(s; \theta)$ . Therefore, while  $\omega$  is fixed, we seek to find parameter  $\theta$  that satisfies the following equation:

$$Q(s, a, \mu(s; \theta); \omega) \approx \sup_{y_a \in \mathcal{Y}} Q(s, a, y_a; \omega). \quad (36)$$

Similar to DDPG, P-DQN adopts an experience replay mechanism and employs target networks to compute the TD target. Network weights  $\omega$  are optimized by minimizing TD loss. The loss functions pertaining to  $\omega$  and  $\theta$  are specified respectively in the sequel.

$$L_t(\omega) = \frac{1}{2} [Q(s_t, a_t, y_{a_t}; \omega) - TD]^2, \quad (37)$$

$$L_t(\theta) = - \sum_{a=0}^{M+1} Q[s_t, a, \mu(s; \theta); \omega_{tg}], \quad (38)$$

where  $TD$  is defined as a  $n$ -step TD target, which can be expressed as

$$TD = \sum_{j=0}^{n-1} \gamma^j r_{t+j} + \gamma^n \max_{a \in \mathcal{A}} Q[s_{t+n}, a, \mu(s_{t+n}; \theta_{tg}); \omega_{tg}]. \quad (39)$$

Algorithm 1 illustrates the proposed P-DQN based scheme. At the beginning of each time slot, each vehicle generates computation task demands with a specified probability. The base station collects the tasks and location information of all vehicles and aggregates these data into environmental state information, which are subsequently broadcast. Each agent makes its decisions based on the environment's state. In addition, transitions are saved in an experience replay buffer. Agents update their network weights using data from the replay buffer, eventually converging on the optimal policy. Following the training phase and subsequent deployment, DRL-based algorithm can respond swiftly to the demands based on observation of  $s_t$ , facilitating dynamic and flexible task offloading and resource allocation in vehicular edge computing scenarios.

---

**Algorithm 1:** P-DQN Based Task Offloading and Resource Allocation Strategy.

---

**Input:** State space  $\mathcal{S}$ ; Action space  $\mathcal{A}$ .

**Output:** Optimal actor network weights  $\omega^*$ ; Optimal parameter network weights  $\theta^*$ .

- 1: Initialize network weights:  $\omega, \theta$ ; target network weights  $\omega_{tg}, \theta_{tg}$ ; replay buffer  $D_i$ , ( $i = 1, 2, \dots, N$ ); mini-batch size of transitions  $N_{batch}$ .
  - 2: **while**  $episodes < EP_{max}$  **do**
  - 3: Initialize a set of vehicles  $\mathcal{N}$ ; a set of edge nodes  $\mathcal{M}$  and coverage range of RSU  $R$ ; computation capacity of MEC server  $F$ ; locations and velocities of vehicles.
  - 4: **for**  $t = 1, 2, \dots, T$  **do**
  - 5: **for** agent  $i = 1, 2, \dots, N$  **do**
  - 6: Observe the local state  $o_i(t)$  from the system state  $s_t$ .
  - 7: Determining resource allocation scheme ( $y_0(t), y_1(t), \dots, y_{M+1}(t)$ ).
  - 8: Select offload decision  $a_i(t) \in \mathcal{A}$  according to  $\varepsilon$ -greedy policy.
  - 9: Choose action  $(a_i(t), y_{a_i}(t))$ .
  - 10: **end for**
  - 11: All agents execute actions, interact with environment, and obtain a reward  $r_t$  and that in next state  $s_{t+1}$ .
  - 12: **for** agent  $i = 1, 2, \dots, N$  **do**
  - 13: Store transitions  $(o_i(t), (a_i(t), y_{a_i}(t)), r_t, o_i(t+1))$  in replay buffer  $D_i$ .
  - 14: **if** the number of transitions in replay buffer is greater than  $N_{batch}$ , **then**
  - 15: Randomly sample mini-batch from  $D_i$ .
  - 16: Based on the loss functions defined in (37) and (38), compute stochastic gradients  $\nabla_{\omega} L_t(\omega)$  and  $\nabla_{\theta} L_t(\theta)$  using sampled data.
  - 17: Update weights by  $\omega_{t+1} \leftarrow \omega_t - \alpha_t \nabla_{\omega} L_t(\omega)$  and  $\theta_{t+1} \leftarrow \theta_t - \beta_t \nabla_{\theta} L_t(\theta)$
  - 18: **end if**
  - 19: **end for**
  - 20: **end for**
  - 21:  $episodes = episodes + 1$
  - 22: **end while**
- 

## V. PERFORMANCE EVALUATION

In this section, simulations are performed to evaluate the effectiveness of the proposed P-DQN-based task offloading and resource allocation algorithm. The simulation scenario is established based on the network model as shown in Fig. 1. We aim to evaluate the P-DQN algorithm's performance across training and inference phases with a particular focus on its convergence properties. First, let us take a look at the training performance under various parameter configurations to determine the optimal setting. Then, we compare the proposed scheme with some well-known benchmark algorithms on training convergence performance and total rewards. Major parameters used in the simulations are listed in Table II.

TABLE II  
MAJOR PARAMETERS USED IN SIMULATIONS

| Parameter                     | Value                       | Description                                                          |
|-------------------------------|-----------------------------|----------------------------------------------------------------------|
| $\Delta t$                    | 1 s                         | Duration of a time slot                                              |
| $B$                           | 30 MHz                      | Uplink bandwidth for RSU                                             |
| $PL$                          | $128.1 + 37.6 \lg \Delta d$ | Path loss model ( $\Delta d$ is the distance from Tx to Rx)          |
| $\sigma^2$                    | -97 dBm                     | Background noise power                                               |
| $L$                           | 1,000 m                     | The length of road segment                                           |
| $R$                           | 250 m                       | Coverage range of each RSU                                           |
| $P$                           | 0.5                         | Probability that a vehicle generates a task at each time slot        |
| $d_i(t)$                      | [8,000, 12,000] bit         | Input data size of a task                                            |
| $\lambda_i(t)$                | $10^5$ cycles/bit           | Service coefficient                                                  |
| $t_i^{finish}(t)$             | [0.9, 1.0] s                | Tasks' maximum delay                                                 |
| $D_{size}$                    | 20,000                      | Experience replay buffer size                                        |
| $N_{batch}$                   | 128                         | Mini-batch size                                                      |
| $lra$                         | $10^{-5}$                   | Learning rate of actor network                                       |
| $lrp$                         | $10^{-6}$                   | Learning rate of parameter network                                   |
| $\gamma$                      | 0.9                         | Discount factor                                                      |
| $\varepsilon_i/\varepsilon_f$ | 1.0/0.001                   | Initial/Final parameter of the $\varepsilon$ -greedy                 |
| $F$                           | 10 GHz                      | Computation capability of MEC server                                 |
| $F^c$                         | 10 GHz                      | Computation capability of cloud server                               |
| $\kappa$                      | $10^{-26}$                  | Energy consumption parameter                                         |
| $\rho^E$                      | 0.15 \$/GHz                 | Price of MEC servers' resource                                       |
| $\rho^c$                      | 0.3 \$/GHz                  | Price of cloud server's resource                                     |
| $P_{max}$                     | 2 W                         | Maximum vehicle transmit power                                       |
| $P_{min}$                     | 1 W                         | Minimum vehicle transmit power                                       |
| $\tau_a/\tau_p$               | 0.1/0.001                   | Soft update parameter for the target actor network/parameter network |
| $w_1, w_2, w_3$               | 1/3                         | Weights of delay, energy consumption, and cost                       |

### A. Parameter Setting

Simulations were conducted based on Python 3.11 and PyTorch 2.0. Simulated network scenario comprises a 1,000-meter one-way road with 20 moving vehicles, each of which is equipped with a computation capability of 1 GHz and can adjust transmit power ranging from 1 to 2 W. Four RSUs are deployed evenly along the road, and each of them is equipped with a MEC server of 10 GHz computation capability. Total uplink bandwidth of each RSU is 30 MHz. At the beginning of each time slot, there is a probability ( $P = 0.5$ ) that a vehicle will initiate a computing task. Input data sizes of these tasks are uniformly distributed from 8,000 to 12,000 bits, and processed at a service coefficient of  $\lambda_i(t) = 100$  kilocycles/b, with the maximum delay per task selected from the range of [0.9, 1] second. With consideration to both the convergence speed and training stability, the experience replay buffer size for P-DQN algorithm is set to 20,000, with a mini-batch size of 128, and the optimizer used is Adam. The soft update parameters for the target actor and target parameter networks are set to  $\tau_a = 0.1$  and  $\tau_p = 0.001$ , respectively. The values of  $w_1, w_2$  and  $w_3$  are all set to 1/3, indicating that delay, energy consumption, and cost are assigned with equal weights. To ensure stronger exploration during early stages of training and gradually shift toward a more greedy strategy, we implemented an annealing schedule for  $\varepsilon$  in

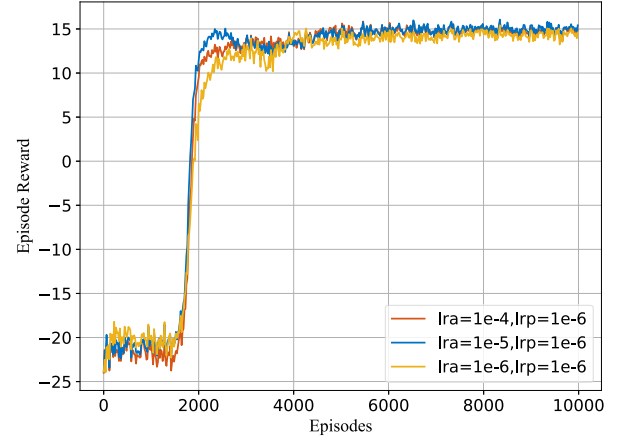


Fig. 5. Episode reward versus episodes under different learning rates of the actor network.

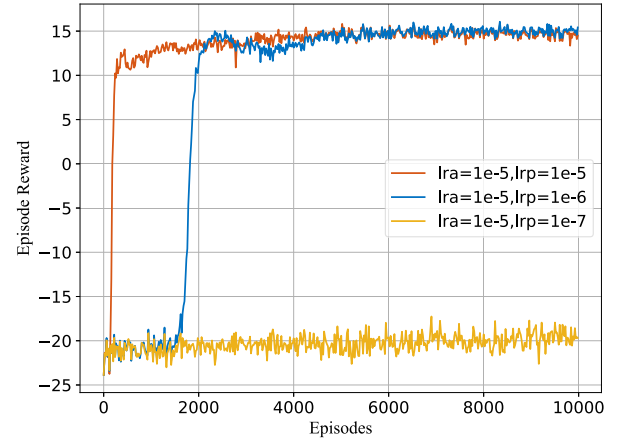


Fig. 6. Episode reward versus episodes under different learning rates of the parameter network.

the P-DQN algorithm. At the beginning of training,  $\varepsilon$  is set to 1 and decreases gradually to 0.001 as the number of episodes increases. The discount factor  $\gamma$ , which reflects the importance of future rewards in the current decision-making, is set to 0.9. The learning rate settings will be elaborated in the next subsection.

### B. Convergence Performance

The effects of learning rates on convergence efficiency of the proposed P-DQN-based algorithm are evaluated first. The simulation results are shown in Figs. 5 and 6, where  $lra$  and  $lrp$  represent the learning rates for the actor and parameter networks, respectively. The results shown in Fig. 5 highlight how a varying  $lra$  value influences the algorithm's reward outputs with an increasing number of episodes. The learning rate of the parameter network is fixed at  $10^{-6}$ , while the actor network's learning rate  $lra$  is varying. Optimal performance, which is characterized by enhanced convergence and maximum reward, is achieved at a learning rate of  $10^{-5}$  for the actor network. Similarly, Fig. 6 depicts the trajectory of total rewards varying with the number of training episodes, where  $lra$  is fixed at  $10^{-5}$  and  $lrp$

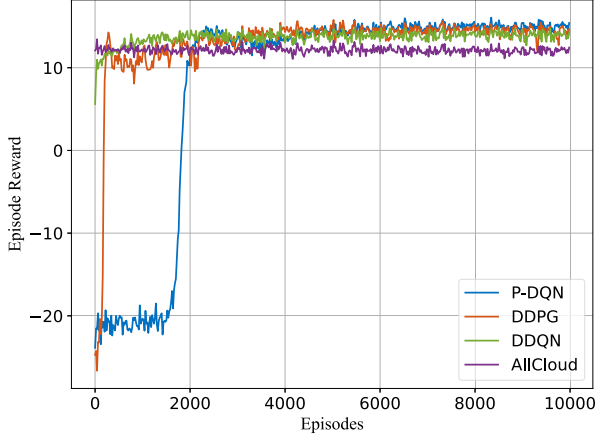


Fig. 7. Episode reward versus episodes with different algorithms.

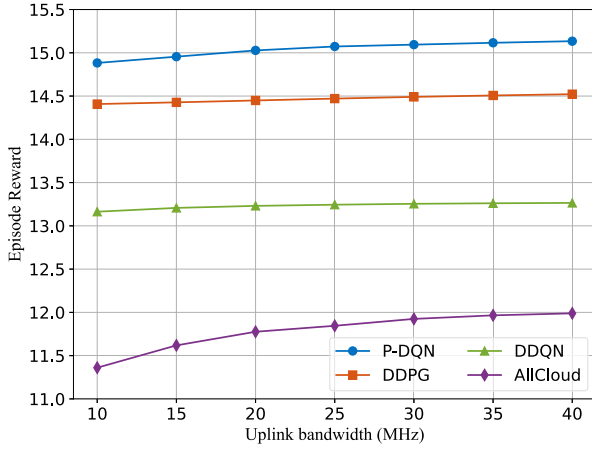
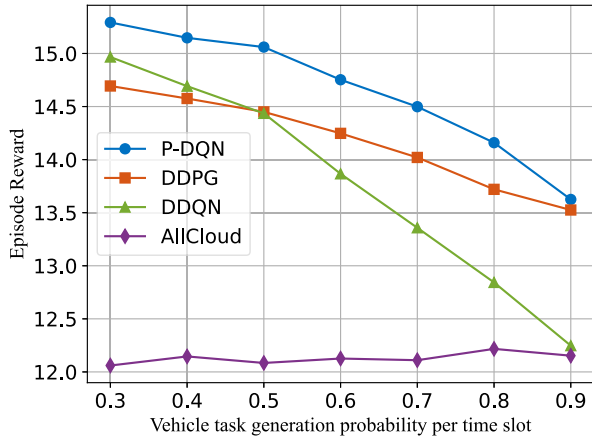
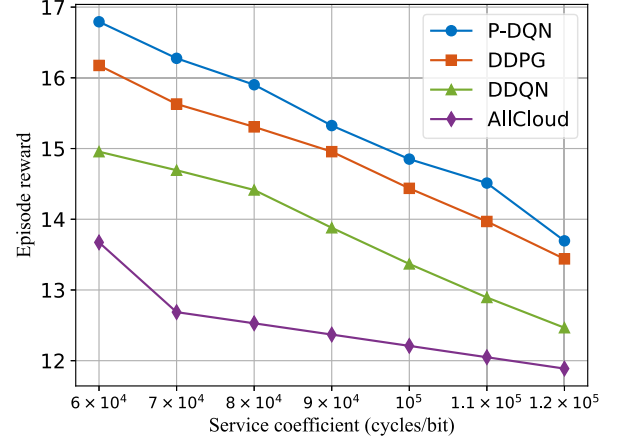
Fig. 8. Episode reward versus total uplink bandwidth of each RSU ( $B$ ).

Fig. 9. Episode reward versus each vehicle's task generation probability.

changes amongst  $10^{-5}$ ,  $10^{-6}$ , and  $10^{-7}$ , respectively. Although the learning rate  $l_{rp}$  of  $10^{-5}$  facilitates a quicker convergence, a lower rate of  $10^{-6}$  results in a higher ultimate reward, suggesting that elevating learning rate may induce a premature convergence at a local optimum. Conversely, an excessively low rate of  $10^{-7}$  impedes training process significantly, thereby weakening the algorithm's capability to converge timely.

Fig. 10. Episode reward versus service coefficient  $\lambda_i(t)$ .

Based on these observations, we fix the learning rates for the actor and parameter networks at  $10^{-5}$  and  $10^{-6}$ , respectively, for subsequent simulations. This setting aims to achieve a balance that can ensure effective learning and the highest reward.

### C. Comparative Analysis

To further verify the performance of the proposed P-DQN-based algorithm, we particularly designed the following three algorithms as the benchmarks for comparative analysis.

- **Double DQN:** It is also called DDQN and represents an enhanced version of DQN algorithm, addressing the issue on overestimation of Q-values in the original DQN. Given that the output of DDQN is discrete, while vehicle transmit power  $p_i(t)$  and computation resource variables  $f_{i,m}^E(t)$ ,  $f_i^c(t)$  are continuous, it necessitates the approximation of continuous parameters with discrete actions [36]. In this work,  $f_{i,m}^E(t)$  and  $f_i^c(t)$  are quantized into ten levels ranging from 0 to  $F$  and from 0 to  $F^c$ , while  $p_i(t)$  is quantized into three levels.
- **DDPG:** As DDPG strategy cannot handle discrete-continuous hybrid action space directly, the same method as described in [39], [47] was applied to relax the action space. This relaxation enables DDPG to output offloading decisions and resource allocation actions simultaneously.
- **AllCloud:** This approach offloads all generated computing tasks to cloud servers via RSUs' relaying and allocates fixed computing resources for them. In addition, transmit power of a vehicle connected to the RSU is fixed at  $P_{max}$ .

Fig. 7 depicts episode reward performance versus the number of training episodes under the proposed P-DQN-based strategy and the three benchmark algorithms, i.e., DDPG, DDQN, and AllCloud. It is observed that, with the three DRL-based algorithms, agents enhance their policies through interaction with environment before attaining convergence. After convergence, all DRL-based schemes exhibit superior performance compared to AllCloud strategy, albeit with some differences in their

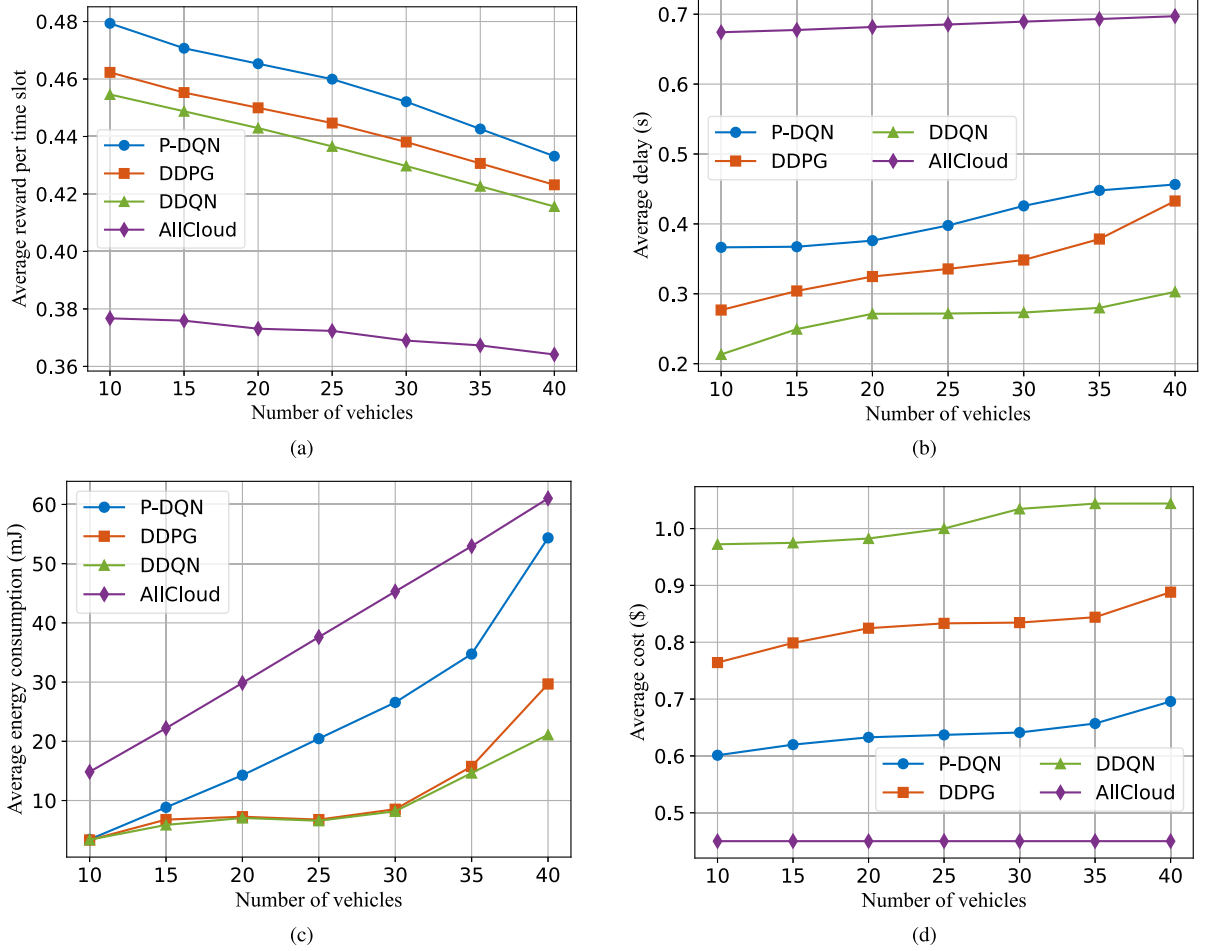


Fig. 11. Average reward, delay, energy consumption, and cost versus the number of vehicles ( $N$ ). (a) Average reward per time slot (b) Average delay (c) Average energy consumption (d) Average cost.

convergence speeds, stability, and ultimate rewards achieved. Specifically, DDQN strategy shows a rapid convergence, stabilizing at around 2,000 episodes and achieving complete convergence by 4,000 episodes. DDPG strategy offers a relatively stable reward within about 4,000 episodes, yet experiencing some performance fluctuations after convergence. P-DQN strategy secures convergence within about 6,000 episodes, getting the highest level of stability and reward. Furthermore, after achieving the convergence, the P-DQN-based algorithm gives an average reward gain of 0.21 and 1.26 over the DDPG and DDQN algorithms, respectively.

Fig. 8 shows episode reward with a varying total uplink bandwidth of each RSU with the proposed P-DQN and three benchmark algorithms. It is seen that reward performance of each algorithm shows an upward trend with an increasing  $B$ . This is reasonable, as increasing total uplink bandwidth of each RSU (i.e., increasing  $B$ ) can increase data rate for vehicles to upload their input data of offloading tasks to RSUs, thereby reducing overall transmission delay. In addition, it is apparent that changes in  $B$  affect the reward performance of AllCloud strategy more significantly. This occurs as AllCloud strategy offloads all tasks to server for execution, while in certain epochs, DRL-based

algorithm may choose to execute tasks locally, depending on the current environmental state. In fact, its local execution delay is independent of  $B$ .

Fig. 9 illustrates episode reward performance of the four algorithms versus each vehicle's task generation probability  $P$ . With an increasing  $P$ , overall network load will gradually increase, finally leading to additional task execution delay. Consequently, this makes agents suffer higher costs to offload tasks to resource-rich cloud servers, resulting in a decrease in system's overall episode rewards. As a fixed strategy, AllCloud's reward performance is influenced solely by  $P$  during the input data upload phase of task offloading, thus maintaining nearly a constant episode reward.

Fig. 10 demonstrates system's episode reward performance versus service coefficient  $\lambda_i(t)$  varying from  $6 \times 10^4$  to  $1.2 \times 10^5$  cycles/bit across the four algorithms. Obviously, increasing  $\lambda_i(t)$  results in a longer task execution time, thus increasing total delay, which finally diminishes overall system reward. Compared to the benchmark schemes, the proposed algorithm optimizes episode rewards by leveraging a parameterized action space, hence showing a superior reward performance.

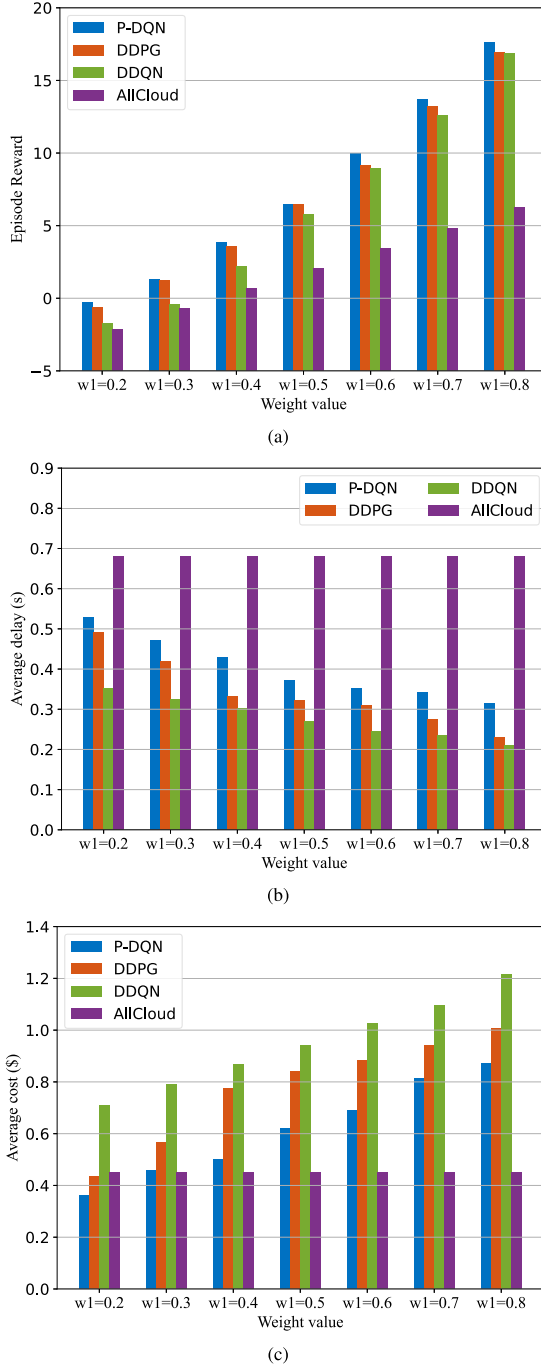


Fig. 12. Average delay, energy consumption, and cost with different values of  $w_1$  and  $w_3$  while  $w_2 = 0$  (i.e.,  $w_3 = 1 - w_1$ ). (a) Episode reward (b) Average delay (c) Average cost.

Fig. 11 illustrates the impact of the number of vehicles on the system's average reward, delay, energy consumption, and cost. Fig. 11(a) shows the average reward per vehicle per time slot, which decreases as the number of vehicles in the network increases. The P-DQN algorithm consistently achieves the best overall performance. It is observed that the three DRL algorithms adopt different strategies to balance various objectives. As the number of vehicles increases, interference within the

system intensifies, and computational resources become more constrained, resulting in an increase in delay, energy consumption, and cost for all three DRL algorithms. Overall, the P-DQN algorithm generally completes computational tasks at a lower cost, while maintaining a moderate delay. In comparison, AllCloud algorithm exhibits a higher delay, while DDQN and DDPG algorithms incur higher costs compared to the other two. Additionally, as shown in Fig. 11(c), when the number of vehicles is small, the average energy consumption of the three DRL algorithms is nearly identical. However, when the number of vehicles exceeds  $N \geq 20$ , the average energy consumption of the P-DQN algorithm increases significantly compared to that of DDPG and DDQN. Given that the P-DQN algorithm exhibits slightly higher delay and lower cost, this may be attributed to the P-DQN algorithm's reliance more heavily on a local computation strategy.

The above results indicate that the three DRL algorithms adopt different trade-off strategies for multiple objectives, which may also be influenced by weight settings. Fig. 12 depicts the influences of different weight settings, i.e.,  $w_1$ ,  $w_2$ , and  $w_3$ , on system's episode reward, average delay, and cost, where the values of  $w_1$  and  $w_3$  vary, while  $w_2$  is fixed as zero, with the constraint  $w_1 + w_2 + w_3 = 1$ . Fig. 12(a), (b), and (c) show system performances on episode reward, average delay, and average cost per time slot, respectively. From Fig. 12(a), we can see that as  $w_1$  increases (or  $w_3$  decreases), episode reward performances of the four different strategies gradually improve. The reason is that the systems with these strategies may select to pay a higher cost for having more computation resources to minimize task execution delay, to respond to a higher delay weight value. Such observation is further confirmed by the results as shown in Fig. 12(b) and (c). However, as AllCloud algorithm allocates fixed resources to all vehicles, it maintains constant average delay and cost as  $w_1$  increases, and its episode reward variation stems mainly from the changes in reward calculation method. In addition, while P-DQN-based algorithm always acquires the highest episode reward, its average delay slightly exceeds that of DDPG and DDQN strategies. On the other hand, P-DQN algorithm requires a relatively lower average cost, compared to the other strategies. This validates that the proposed scheme can balance different aspects of system performances for attaining a superior effectiveness.

## VI. CONCLUSION

In this paper, we investigated the issues on multi-vehicle joint task offloading and resource allocation in cloud-assisted vehicular MEC networks. RSUs equipped with MEC servers are taken as edge nodes. Computation-intensive and delay-sensitive tasks generated by vehicles can be offloaded to edge nodes or cloud server through binary offloading. Based on the system model, we formulated a task offloading and resource allocation optimization problem, aiming to minimize overall delay, energy consumption, and cost. The formulated problem was converted to an MDP and solved via DRL-based strategies. In particular, to tackle hybrid action space issues in binary offloading, a parameterized action space was introduced, based on which a

P-DQN algorithm for real-time task offloading and resource allocation scheme was designed. Simulation results validated the effectiveness of the proposed algorithm in various network scenarios and its advantages over three well-known benchmark algorithms, i.e., DDQN, DDPG, and AICloud.

## REFERENCES

- [1] F. Tang, Y. Kawamoto, N. Kato, and J. Liu, "Future intelligent and secure vehicular network toward 6G: Machine-learning approaches," *Proc. IEEE*, vol. 108, no. 2, pp. 292–307, Feb. 2020.
- [2] S. Gyawali, S. Xu, Y. Qian, and R. Q. Hu, "Challenges and solutions for cellular based V2X communications," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 1, pp. 222–255, Firstquarter 2021.
- [3] H. Zhang and X. Lu, "Vehicle communication network in intelligent transportation system based on Internet of Things," *Comput. Commun.*, vol. 160, pp. 799–806, Jul. 2020.
- [4] Z. Wang, K. Han, and P. Tiwari, "Augmented reality-based advanced driver-assistance system for connected vehicles," in *Proc. 2020 IEEE Int. Conf. Syst. Man Cybern.*, Toronto, ON, Canada, 2020, pp. 752–759.
- [5] S. K. Sood et al., "Smart vehicular traffic management: An edge cloud centric IoT based framework," *Internet Things*, vol. 14, Jun. 2021, Art. no. 100140.
- [6] Y. C. Hu, M. Patel, D. Sabella, N. Sprecher, and V. Young, "Mobile edge computing - A key technology towards 5G," *ETSI White Paper*, vol. 11, no. 11, pp. 1–16, Sep. 2015.
- [7] D. Katere, D. Perino, J. Nurmi, M. Warnier, M. Janssen, and A. Y. Ding, "A survey on approximate edge AI for energy efficient autonomous driving services," *IEEE Commun. Surveys Tuts.*, vol. 25, no. 4, pp. 2714–2754, Fourth Quarter 2023.
- [8] B. Ji et al., "A vision of IoV in 5G HetNets: Architecture, key technologies, applications, challenges, and trends," *IEEE Netw.*, vol. 36, no. 2, pp. 153–161, Mar./Apr. 2022.
- [9] W. Duan, J. Gu, M. Wen, G. Zhang, Y. Ji, and S. Mumtaz, "Emerging technologies for 5G-IoV networks: Applications, trends and opportunities," *IEEE Netw.*, vol. 34, no. 5, pp. 283–289, Sep./Oct. 2020.
- [10] A. Hammoud, H. Sami, A. Mourad, H. Otrok, R. Mizouni, and J. Bentahar, "AI, blockchain, and vehicular edge computing for smart and secure IoV: Challenges and directions," *IEEE Internet Things Mag.*, vol. 3, no. 2, pp. 68–73, Jun. 2020.
- [11] L. Liu, C. Chen, Q. Pei, S. Maharjan, and Y. Zhang, "Vehicular edge computing and networking: A survey," *Mobile Netw. Appl.*, vol. 26, pp. 1145–1168, Jul. 2021.
- [12] L. Br hon-Grataloup, R. Kacimi, and A.-L. Beylot, "Mobile edge computing for V2X architectures and applications: A survey," *Comput. Netw.*, vol. 206, Apr. 2022, Art. no. 108797.
- [13] A. Y. Alhilal, B. Finley, T. Braud, D. Su, and P. Hui, "Street smart in 5G: Vehicular applications, communication, and computing," *IEEE Access*, vol. 10, pp. 105631–105656, 2022.
- [14] X. Jiang, F. R. Yu, T. Song, and V. C. M. Leung, "Resource allocation of video streaming over vehicular networks: A survey, some research issues and challenges," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 7, pp. 5955–5975, Jul. 2022.
- [15] M. K. Hasan, N. Jahan, M. Z. A. Nazri, S. Islam, M. A. Khan, and A. I. Alzahrani, "Federated learning for computational offloading and resource management of vehicular edge computing in 6G-V2X network," *IEEE Trans. Consum. Electron.*, vol. 70, no. 1, pp. 3827–3847, Feb. 2024.
- [16] H. Liu, H. Zhao, L. Geng, and W. Feng, "A policy gradient based offloading scheme with dependency guarantees for vehicular networks," in *Proc. IEEE Glob. Commun. Conf.*, Dec. 2020, pp. 1–6.
- [17] H. Ke, H. Wang, W. Sun, and H. Sun, "Adaptive computation offloading policy for multi-access edge computing in heterogeneous wireless networks," *IEEE Trans. Netw. Service Manag.*, vol. 19, no. 1, pp. 289–305, Mar. 2022.
- [18] W. Fan et al., "Joint task offloading and resource allocation for vehicular edge computing based on V2I and V2V modes," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 4, pp. 4277–4292, Apr. 2023.
- [19] Q. Liu, R. Luo, and Q. Liu, "Mobility-aware computation offloading for cloud-assisted mobile edge computing in vehicular networks," in *Proc. IEEE 96th Veh. Technol. Conf.*, London, U.K., 2022, pp. 1–7.
- [20] Q. Luo, C. Li, T. H. Luan, W. Shi, and W. Wu, "Self-learning based computation offloading for internet of vehicles: Model and algorithm," *IEEE Trans. Wireless Commun.*, vol. 20, no. 9, pp. 5913–5925, Sep. 2021.
- [21] W. Feng et al., "Latency minimization of reverse offloading in vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 71, no. 5, pp. 5343–5357, May 2022.
- [22] R. Yadav, W. Zhang, O. Kaiwartya, H. Song, and S. Yu, "Energy-latency tradeoff for dynamic computation offloading in vehicular fog computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 14198–14211, Dec. 2020.
- [23] Z. Wang, J. Du, C. Jiang, Y. Ren, and X.-P. Zhang, "UAV-Assisted target tracking and computation offloading in USV-Based MEC networks," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 11389–11405, Dec. 2024.
- [24] J. Du, C. Jiang, A. Benslimane, S. Guo, and Y. Ren, "SDN-Based resource allocation in edge and cloud computing systems: An evolutionary stackelberg differential game approach," *IEEE/ACM Trans. Netw.*, vol. 30, no. 4, pp. 1613–1628, Aug. 2022.
- [25] M. Huang, Z. Shen, and G. Zhang, "Joint spectrum sharing and V2V/V2I target tracking for vehicular edge computing networks based on coalition formation game," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 9, pp. 11918–11934, Sep. 2024.
- [26] L. Zhao, S. Huang, D. Meng, B. Liu, Q. Zuo, and V. C. M. Leung, "Stackelberg-game-Based dependency-aware task offloading and resource pricing in vehicular edge networks," *IEEE Internet Things J.*, vol. 11, no. 19, pp. 32337–32349, Oct. 2024.
- [27] W. Fan et al., "Game-based task offloading and resource allocation for vehicular edge computing with edge-edge cooperation," *IEEE Trans. Veh. Technol.*, vol. 72, no. 6, pp. 7857–7870, Jun. 2023.
- [28] Q. Luo, S. Hu, C. Li, G. Li, and W. Shi, "Resource scheduling in edge computing: A survey," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 4, pp. 2131–2165, Fourth Quarter 2021.
- [29] A. Feriani and E. Hossain, "Single and multi-agent deep reinforcement learning for AI-Enabled wireless networks: A tutorial," *IEEE Commun. Surveys Tuts.*, vol. 23, no. 2, pp. 1226–1252, Second Quarter 2021.
- [30] J. Liu et al., "RL/DRL meets vehicular task offloading using edge and vehicular cloudlet: A survey," *IEEE Internet Things J.*, vol. 9, no. 11, pp. 8315–8338, Jun. 2022.
- [31] N. A. Khalek, D. H. Tashman, and W. Hamouda, "Advances in machine learning-driven cognitive radio for wireless networks: A survey," *IEEE Commun. Surveys Tuts.*, vol. 26, no. 2, pp. 1201–1237, Second Quarter 2024.
- [32] M. Hu, L. Zhuang, D. Wu, Y. Zhou, X. Chen, and L. Xiao, "Learning driven computation offloading for asymmetrically informed edge computing," *IEEE Trans. Parallel Distrib. Syst.*, vol. 30, no. 8, pp. 1802–1815, Aug. 2019.
- [33] W. Lv et al., "Microservice deployment in edge computing based on deep Q learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 11, pp. 2968–2978, Nov. 2022.
- [34] J. Wu et al., "Resource allocation for delay-sensitive vehicle-to-multi-edges (V2Es) communications in vehicular networks: A multi-agent deep reinforcement learning approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 1873–1886, Second Quarter 2021.
- [35] Y. Liu, H. Yu, S. Xie, and Y. Zhang, "Deep reinforcement learning for offloading and resource allocation in vehicle edge computing and networks," *IEEE Trans. Veh. Technol.*, vol. 68, no. 11, pp. 11158–11168, Nov. 2019.
- [36] L. Wang and G. Zhang, "Joint service caching, resource allocation and computation offloading in three-tier cooperative mobile edge computing system," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 6, pp. 3343–3353, Nov./Dec. 2023.
- [37] H. Peng and X. Shen, "Deep reinforcement learning based resource management for multi-access edge computing in vehicular networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 4, pp. 2416–2428, Fourth Quarter 2020.
- [38] H. Yang, Z. Wei, Z. Feng, X. Chen, Y. Li, and P. Zhang, "Intelligent computation offloading for MEC-Based cooperative vehicle infrastructure system: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 71, no. 7, pp. 7665–7679, Jul. 2022.
- [39] Z. Xue, C. Liu, C. Liao, G. Han, and Z. Sheng, "Joint service caching and computation offloading scheme based on deep reinforcement learning in vehicular edge computing systems," *IEEE Trans. Veh. Technol.*, vol. 72, no. 5, pp. 6709–6722, May 2023.
- [40] G. Ma, X. Wang, M. Hu, W. Ouyang, X. Chen, and Y. Li, "DRL-Based computation offloading with queue stability for vehicular-cloud-assisted mobile edge computing systems," *IEEE Trans. Intell. Veh.*, vol. 8, no. 4, pp. 2797–2809, Apr. 2023.

- [41] L. Geng, H. Zhao, J. Wang, A. Kaushik, S. Yuan, and W. Feng, "Deep-reinforcement-Learning-Based distributed computation offloading in vehicular edge computing networks," *IEEE Internet Things J.*, vol. 10, no. 14, pp. 12416–12433, Jul. 2023.
- [42] Z. Chen, J. Du, G. Yang, C. Jiang, and Z. Han, "Multi-task driven user association and resource allocation in in-vehicle networks," in *Proc. 2023 IEEE Glob. Commun. Conf.*, Kuala Lumpur, Malaysia, 2023, pp. 01–06.
- [43] J. Du, T. Lin, C. Jiang, Q. Yang, C. F. Bader, and Z. Han, "Distributed foundation models for multi-modal learning in 6G wireless networks," *IEEE Wireless Commun.*, vol. 31, no. 3, pp. 20–30, Jun. 2024.
- [44] Y. Jang, J. Na, S. Jeong, and J. Kang, "Energy-efficient task offloading for vehicular edge computing: Joint optimization of offloading and bit allocation," in *Proc. IEEE 91st Veh. Technol. Conf.*, Antwerp, Belgium, 2020, pp. 1–5.
- [45] J. Xiong et al., "Parametrized deep Q-networks learning: Reinforcement learning with discrete-continuous hybrid action space," 2018, *arXiv:1810.06394*.
- [46] T. P. Lillicrap et al., "Continuous control with deep reinforcement learning," 2015, *arXiv:1509.02971*.
- [47] M. Hausknecht and P. Stone, "Deep reinforcement learning in parameterized action space," 2015, *arXiv:1511.04143*.



communications, intelligent connected-vehicles, satellite communication networks, and underwater-overwater cooperative communications and networks.



**Ruofei Ma** (Member, IEEE) received the B.Sc., M.Sc., and Ph.D. degrees in information and communication engineering from Harbin Institute of Technology, Harbin, China, in 2008, 2010, and 2014, respectively. From 2015 to 2016, he was a Postdoctoral Researcher with the Department of Engineering Science, National Cheng Kung University, Taiwan. He is currently an Associate Professor with the Department of Communication Engineering, Harbin Institute of Technology, Weihai, China. His research interests include device-to-device (D2D) communications, intelligent connected-vehicles, satellite communication networks, and underwater-overwater cooperative communications and networks.

**Jingyang Zhou** received the B.Sc. degree from Hohai University, Nanjing, China, in 2022. He is currently working toward the master's degree in information and communication engineering with the Harbin Institute of Technology (HIT), Weihai, China. His research interests include vehicular communications and networks, mobile edge computing, and deep reinforcement learning.



**Ruisong Wang** received the B.Sc. degree in information and computing science from the Harbin Institute of Technology, Weihai, China, in 2018. He is currently working toward the Ph.D. degree in information and communication engineering from the Harbin Institute of Technology, Harbin, China. His research interests include underwater-overwater cooperative communications, efficient resource allocation, and satellite networks.



**Hsiao-Hwa Chen** (Life Fellow, IEEE) is currently a Distinguished Professor with the Department of Engineering Science, National Cheng Kung University, Taiwan. He received the B.Sc. and M.Sc. degrees from Zhejiang University, Hangzhou, China, and the Ph.D. degree from the University of Oulu, Oulu, Finland, in 1982, 1985, and 1991, respectively. He has authored or coauthored more than 400 technical papers in major international journals and conferences, six books, and more than ten book chapters in the areas of communications. He was the recipient of the Best Paper Award of IEEE Systems Journal in 2021 and the IEEE 2016 Jack Neubauer Memorial Award. He was the TPC Chair for IEEE Globecom 2019. He is also the founding Editor-in-Chief of Wiley's *Security and Communication Networks* Journal. From 2012 to 2015, he was the Editor-in-Chief of IEEE WIRELESS COMMUNICATIONS. From 2015 to 2016, he was an elected Member-at-Large of IEEE ComSoc. He is also a Fellow of IET/BCS/AAIA.



**Gongliang Liu** (Member, IEEE) received the B.Sc. degree in measuring and control technology and instrumentations, and the M.Sc. and Ph.D. degrees in information and communication engineering from Harbin Institute of Technology (HIT), Harbin, China, in 2001, 2003, and 2007, respectively. From 2015 to 2016, he was a Visiting Scholar with the University of British Columbia, Vancouver, BC, Canada. He is currently a Professor with HIT, Weihai, China. His research interests are wireless communications and networks, satellite communications, and underwater communications.